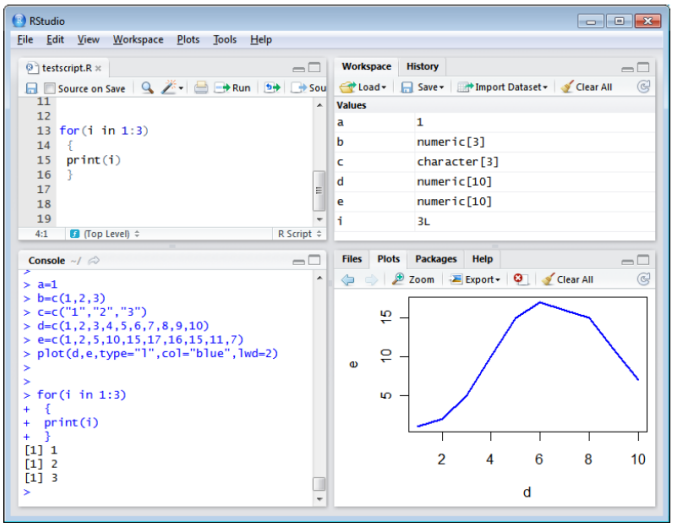
Data Preparation for Business Analytics Session 1 Handout

RStudio

Data types: Variable, Vector and factor, Matrix, Data Frame, List…

RStudio layout

The RStudio interface consists of several windows (see Figure below).



Bottom left: console window (also called command window). Here you can type simple commands after the > prompt and R will then execute your command. This is the most important window, because this is where R actually does stuff.

Top left: editor window (also called script window). Collections of commands (scripts) can be edited and saved. When you don't get this window, you can open it with File -> New -> R script. Just typing a command in the editor window is not enough, it has to get into the command window before R executes the command. If you want to run a line from the script window (or the whole script), you can click Run or press CTRL+ENTER to send it to the command window.

Top right: workspace / history window. In the workspace window you can see which data and values R has in its memory. You can view and edit the values by clicking on them. The history window shows what has been typed before.

Bottom right: Files / plots / packages / help window. Here you can open files, view plots (also previous plots), install and load packages or use the help function. You can change the size of the windows by dragging the grey bars between the windows.

Working directory

Your working directory is the folder on your computer in which you are currently working. When you ask R to open a certain file, it will look in the working directory for this file, and when you tell R to save a data file or figure, it will save it in the working directory.

Before you start working, please set your working directory to where all your data and script files are or should be stored. Type in the command window: setwd("directoryname").

Make sure that the slashes are forward slashes and that you don't forget the apostrophes. R is case sensitive, so make sure you write capitals where necessary. Within RStudio you can also go to Tools / Set working directory.

Libraries

R can do many statistical and data analyses. They are organized in so-called packages or libraries. With the standard installation, most common packages are installed. To get a list of all installed packages, go to the packages window or type library() in the console 2 window. If the box in front of the package name is ticked, the package is loaded (activated) and can be used.

There are many more packages available on the R website. If you want to install and use a package, you should:

Install the package: click install packages in the packages window and type geometry or type install.packages("tidyverse") in the command window.

Load the package: check box in front of tidyverse or type library(tidyverse) in the command window.

To remove all variables from R's memory,

type
rm(list=ls())

Functions

If you would like to compute the mean of all the elements in the vector b from the example above, you could type
(3+4+5)/3

When you use a function to compute a mean, you'll type:
b <- c(3, 4, 5)
mean(b)

If you need help (e.g. the syntax of a function):

help()

Just using Google can also be very productive.

Scripts

R is an interpreter that uses a command line-based environment. This means that you have to type commands, rather than use the mouse and menus. This has the advantage that you do not always have to retype all commands. You can store your commands in files, the so-called scripts. These scripts have typically file names with the extension .R.

You can open an editor window to edit these files by clicking File and New or Open file....

You can run (send to the console window) part of the code by selecting lines and pressing CTRL+ENTER or click Run in the editor window. If you do not select anything, R will run the line your cursor is on.

You can always run the whole script with the console command **source**, so e.g. for the script in the file foo.R you type: > source("foo.R")

You can also click Run all in the editor window or type CTRL+SHIFT + S to run the whole script at once.

The primary data types in R:

Numeric
- Description: Represents numbers, which can be integers or real numbers (decimals).
- Examples: `42`, `3.14`, `-100`
- Note: By default, numbers in R are treated as double precision (real numbers). To explicitly define an integer, you can use the `L` suffix (e.g., `42L`).

Integer
- Description: Represents whole numbers. Although R automatically treats numbers as numeric (double), you can force a number to be an integer.
- Examples: `42L`, `-10L`
- Note: Use `L` after a number to indicate it's an integer. Otherwise, R treats it as numeric.

Character
- Description: Represents text strings.
- Examples: `"Hello"`, `"R language"`, `'A'`
- Note: Character data is always enclosed in either double (`" "`) or single (`' '`) quotes.

Logical
- Description: Represents boolean values, used to indicate `TRUE` or `FALSE`.
- Examples: `TRUE`, `FALSE`
- Note: Logical values are typically used in conditional statements and are case-sensitive (`True` and `true` are not the same as `TRUE`).

Complex
- Description: Represents complex numbers with real and imaginary parts.
- Examples: `3 + 4i`, `5i`
- Note: The imaginary part is denoted by `i`. Complex numbers are less commonly used in everyday data analysis.

Examples

```
x <- 42        # numeric
y <- 42L       # integer
z <- "Hello"   # character
flag <- TRUE   # logical
cnum <- 3 + 4i  # complex
```

is.xxxx and *as.xxxx* are useful tricks to check data type and transform.

Variables

Naming requirements:
1. Start with a Letter or a Dot (not followed by a number):
- Variable names must start with a letter (a-z, A-Z) or a dot (`.`).
- If a variable name starts with a dot, it should not be followed immediately by a number.

2. Contain Only Letters, Numbers, Dots, and Underscores:
- After the first character, a variable name can contain letters, numbers (0-9), dots (`.`), and underscores (`_`).
- Example: `my_variable1`, `data.frame`, and `Var_2`.

3. Case Sensitivity:
- R is case-sensitive, so `varName`, `VarName`, and `VARNAME` would be considered different variables.

4. Avoid Reserved Words:
- Variable names should not be reserved words in R (e.g., `if`, `else`, `TRUE`, `FALSE`, `NULL`, `for`, `function`, etc.). Attention to potential errors!

5. Length Limit:
- There isn't a strict limit on the length of variable names in R, but overly long names can make your code harder to read and maintain.

6. No Spaces or Special Characters:
- Spaces and special characters are not allowed in variable names.

7. Avoid Starting with Numbers:
- Although variable names can contain numbers, they should not start with them. For example, `var1` is valid, but `1var` is not.

Examples of Valid Variable Names:
- `myVariable`
- `data.frame`
- `.hiddenVar`
- `var_123`

Examples of Invalid Variable Names:
- `1stVar` (starts with a number)
- `my-var` (contains a special character `-`)
- `NULL` (reserved word)

Data Structures:

- Vectors: Ordered collections of elements of the same type (e.g., c(1, 2, 3) for numeric).
- Factors: Categorical variables.
- Lists: Collections of elements of different types.
- Matrices: 2D arrays with elements of the same type.
- Data Frames: Table-like structures where each column can hold different types of data.

Vectors

When you digit *a* < −3 in the console you create an object containing one element only. Suppose you want to create an object containing more than one element. For example, suppose you want to create a vector containing a column (or row) of elements. Imagine this vector *v* = [2, 6, 1, 3, 11]. You can create this:

```
##The command below creates a vector (a column of numbers)
v<-c(2,6,1,3,11)
v
## [1] 2 6 1 3 11
# If you want to create a row containing the same numbers
# You can transpose the vector v using the command t().
myrow<-t(v)
myrow
## [,1] [,2] [,3] [,4] [,5]
## [1,] 2 6 1 3 11
```

Note that the command t() allows transposing a vector or a matrix (this will be discussed below). The vector v is identified using [1] followed by the numbers.

Another way to create a vector is create a sequence of number (in a column) as follows:

```
##The command below creates a vector (a column of numbers)
seq(from=-4, to=5,by=1.5)
## [1] -4.0 -2.5 -1.0 0.5 2.0 3.5 5.0
## if you do not specify by= it is assumed by 1. as shown here
seq(from=-4, to=5)
## [1] -4 -3 -2 -1 0 1 2 3 4 5
## If you want a sequence with a specific length than you have to do:
seq(4, 5, length.out = 5)
## [1] 4.00 4.25 4.50 4.75 5.00
```

Another way to create a vector is create a repetition of numbers (in a column) as follows:

```
##The command below creates a vector (a column of numbers) repeating 5 times the number 2.
rep(2,5)
## [1] 2 2 2 2 2
rep(3,4)
## [1] 3 3 3 3
```

Factors

Factors are tricky vectors; they have double faces: behind the labels for the categorical variable, there are numbers (see storing needs and use in models).

```
gender <- c("male", "female", "male", "male", "male", "female")
genderf <- factor(gender)
gender
```

```
[1] male   female male   male   male   female
Levels: female male
```

They have their ordered version, when the values have a specific order.

```
edu <- c("h", "h", "h", "u", "u", "p", "p", "u", "h", "p")
eduord <- ordered(edu, levels=c("h", "u", "p"))
eduord
```

```
[1] h h h u u p p u h p
Levels: h < u < p
```

The same with labels:

```
eduord <- ordered(edu, levels=c("h", "u", "p"), labels = c("high school", "undergraduate", "postgraduate"))
eduord
```

```
[1] high school   high school   high school   undergraduate undergraduate
postgraduate  postgraduate  undergraduate
 [9] high school   postgraduate
Levels: high school < undergraduate < postgraduate
```

Matrices

Finally, a vector can be considered as a column of a row of a matrix. The latter can be created in R as follows:

```
##The command below creates a 3 by 3 matrix repeating 4 across each element of the matrix.
matrix(4,3,3)
## [,1] [,2] [,3]
## [1,] 4 4 4
## [2,] 4 4 4
## [3,] 4 4 4
##The command below creates a 2 by 2 matrix with the elements of a vector.
matrix(c(11,5,9,2),2,2)
## [,1] [,2]
## [1,] 11 9
## [2,] 5 2
```

```
##The command below creates a vector matrix with the elements of a vector.
matrix(c(1,2,3,4),4,1)
## [,1]
## [1,] 1
## [2,] 2
## [3,] 3
## [4,] 4
##The command below creates a vector matrix with the elements of a vector.
matrix(c(1,2,3,4),1,4)
## [,1] [,2] [,3] [,4]
## [1,] 1 2 3 4
```

You probably noted that, for example [3,], identifies the third row while [,2] identifies the second column. More generally, don't forget the following commands that are needed to extract data from a matrix or a vector.

x[n]: the n^{th} element of a vector
x[m:n]: the m^{th} to nth element
x[c(k,m,n)]: specific elements
x[x>m & x<n]: elements between m and n
[i,j]: element at i^{th} row and j^{th} column
[i,]: row i in a matrix
[,j]: column j in a matrix
x[-c(3,5)]: x excluding the third and fifth elements

```
v1<-c(21,5,2,15)
v2<-c(-3,-6,1,-7)
v3<-c(102,10,-13,4)
x<-matrix(cbind(v1,v2,v3),4,3)
x
## [,1] [,2] [,3]
## [1,] 21 -3 102
## [2,] 5 -6 10
## [3,] 2 1 -13
## [4,] 15 -7 4
x[2] #the nth element of a vector
## [1] 5
x[4:7] ### the mth to nth element
## [1] 15 -3 -6 1
x[c(2,9,4)]### specific elements
## [1] 5 102 15
x[x>0&x<12]### : elements between m and n
## [1] 5 2 1 10 4
x[[11]]### : idem
## [1] -13
x[3,2] ### : element at ith row and jth column
## [1] 1
x[2,]### : row i in a matrix
## [1] 5 -6 10
```

```
x[3:4,1:2] ### : submatrix
## [,1] [,2]
## [1,] 2 1
## [2,] 15 -7
v3[-c(1,2,4)] ##excluding elements
## [1] -13
```

If you want to name the columns (or the rows) of a matrix you can do (but in this case a data frame can be better):

```
colnames(x)<-c("A","B","C")
rownames(x)<-c("F","H","P","Z")
x
## A B C
## F 21 -3 102
## H 5 -6 10
## P 2 1 -13
## Z 15 -7 4
```

You cannot add/subtract/divide vectors of different dimensions. You can multiply vectors with different dimensions by using the symbol %*%, it is the matrix multiplication in math.

```
v1<-c(3,2,6)
v2<-c(-2,-1)
v1%*%t(v2)
## [,1] [,2]
## [1,] -6 -3
## [2,] -4 -2
## [3,] -12 -6
```

For matrices you can add/subtract/multiply/divide vectors with the same dimensions. You cannot add/subtract/divide matrices of different dimensions. You can multiply matrices with different dimensions such as m1%*%m2 ONLY IF the number of columns of m1 corresponds to the number of rows of m2:

```
m1<-matrix(rnorm(6),2,3)
m2<-matrix(rnorm(9),3,3)
m1%*%m2
## [,1] [,2] [,3]
## [1,] -1.279957 0.7353306 0.9001165
## [2,] -1.554485 -0.1157825 1.7718876
```

For matrices the concept of division is linked to the concept of inverse matrix. The matrix inverse is calculated with this *solve*(*m*). You can do m1%*%solve(m2) ONLY IF m2 is a square matrix *number of rows = number of columns*.

```
set.seed(123)
m1<-matrix(rnorm(6),1,2)
m2<-matrix(rnorm(4),2,2)
m1%*%solve(m2)
## [,1] [,2]
## [1,] 0.0385414 0.4570846
```

Here note that m1%*%solve(m2) is something like *m1/m2* but this is a matrix algebra concept that, for the purpose of this course, we do not need to develop more.

Data frames

A data frame is kind of similar to a matrix where the columns have a name, however, columns may have different data types. This is the way you create it:

```
mydatafr<-data.frame(cbind(c(2,1,5),c(22,-4,11)))
colnames(mydatafr)<-c("Col1","Col2")
mydatafr
## Col1 Col2
## 1 2 22
## 2 1 -4
## 3 5 11
```

You can add (or remove) columns like this:

```
mydatafr$Col3<-c("Mark","David","Erika")
mydatafr
```

If you simply put vectors to data.frame(), they will keep their names and data type. In this case, you don't need cbind(). However, it does not work well with vectors of different length.

Lists

Suppose that you have several objects of different types, and you want to collect them in a list (one object). You can do that using the function *list* as follows:

```
a<-3
b<-c(2,6)
c<-matrix(rnorm(4),2,2)
d<-rep(4,3)
mylist<-list(a,b,c,d)
mylist
## [[1]]
## [1] 3
##
## [[2]]
## [1] 2 6
```

```
##
## [[3]]
## [,1] [,2]
## [1,] 1.2240818 0.4007715
##
## [[4]]
## [1] 4 4 4
```

You can now recall the elements of the list as follows: 2, 6 this is the second element.

```
mylist[2]
##[[1]]
##[1] 2 6
```

If you want to remove some elements of a list, you can do this:

```
mylist[-c(2,4)]
## [[1]]
## [1] 3
##
## [[2]]
## [,1] [,2]
## [1,] 1.2240818 0.4007715
## [2,] 0.3598138 0.1106827

#You can also eliminate an element using this:
print("This is another example")
## [1] "This is another example"
mylist[4]<-NULL
mylist
## [[1]]
## [1] 3
##
## [[2]]
## [1] 2 6
##
## [[3]]
## [,1] [,2]
## [1,] 1.2240818 0.4007715
## [2,] 0.3598138 0.1106827
```